their more complete brothers and sisters.

For example, the partial to render a blog entry might be stored in the file `_article.html.erb` in the normal views directory, `app/views/blog`:

```
<div class="article">
  <div class="articleheader">
    <h3><%= article.title %></h3>
  </div>

  <div class="articlebody">
    <%= article.body %>
  </div>
</div>
```

Other templates use the `render(partial:)` method to invoke this:

```
<%= render(partial: "article", object: @an_article) %>
<h3>Add Comment</h3>
. . .
```

The `:partial` parameter to `render` is the name of the template to render (but without the leading underscore). This name must be both a valid filename and a valid Ruby identifier (so `a-b` and `20042501` are not valid names for partials). The `:object` parameter identifies an object to be passed into the partial. This object will be available within the template via a local variable with the same name as the template. In this example, the `@an_article` object will be passed to the template, and the template can access it using the local variable `article`. That's why we could write things such as `article.title` in the partial.

You can set additional local variables in the template by passing `render` a `:locals` parameter. This takes a hash where the entries represent the names and values of the local variables to set:

```
render(partial: 'article',
       object:  @an_article,
       locals:  { authorized_by: session[:user_name],
                  from_ip:       request.remote_ip })
```

Partials and Collections